

从 PLC 到 PC 控制系统的转型实战课程

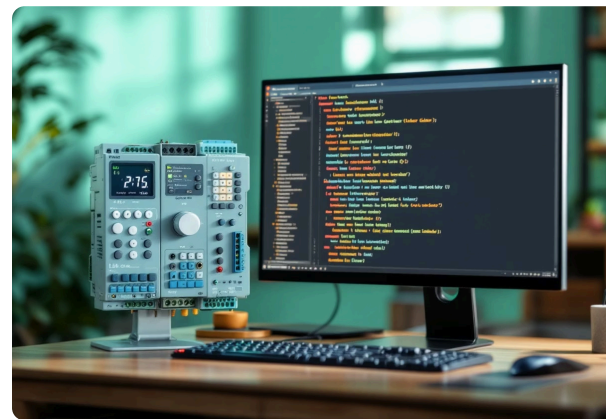
第6课：面向对象程序设计_3 (OOP: Object-Oriented Programming)

物件导向程式设计_3

讲师: [May] | 课程时长: 50 分钟

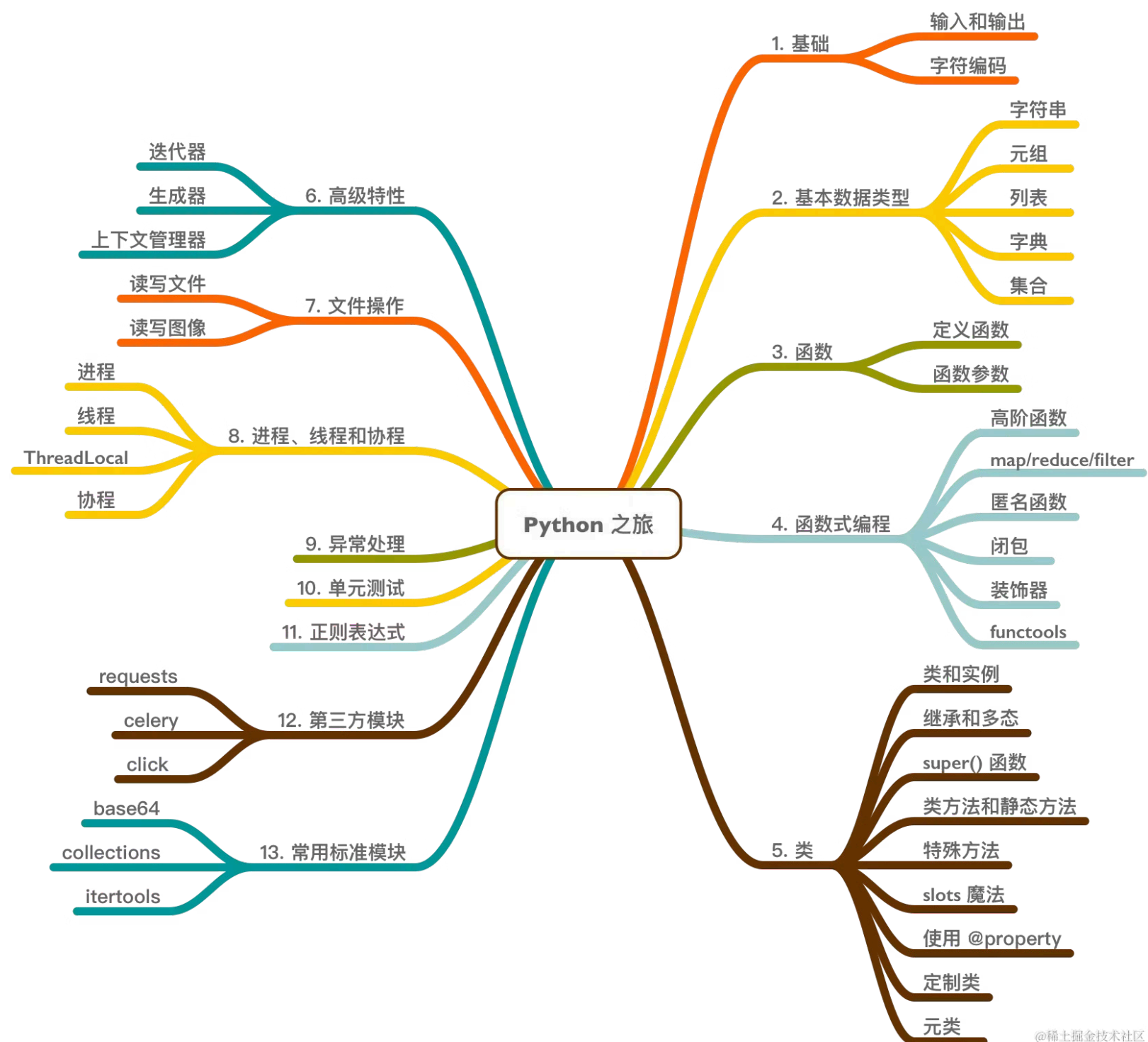
OOP-Based Control Programming

Instructor: [May] | Class Duration: 50 Minutes



从传统工业控制到现代软件控制的转型之旅

复习



@稀土掘金技术社区

类 (Class)

类的成员包括属性 (attribute) 与方法 (method) 两种。例子：

```
class MyClass:
    """
    This is a class that can meow!
    """
    animal = "cat" # An attribute
    def talk(self): # A method
        return "Meow"

# An instance of the class
a = MyClass()
print(a.animal, a.talk())
cat Meow
```

上例中的 `self` 表示类的实例，所有类内部的方法都需要把 `self` 放在首位。

构造函数: `__init__()`

类的构造函数是 `__init__()`（左右均为双下划线），用于初始化实例。在声明实例时，该函数自动被调用。

```
class MyClass2:  
    def __init__(self, animal="cat"):  
        self.animal = animal
```

```
a = MyClass2("dog")  
a.animal  
-> 'dog'
```

Python 私有属性语法

属性写法	类型	是否可在类外访问	用途和语义
self.color	公有属性	✅ 可以	默认公开接口，供外部正常访问和修改
self._status	受保护属性	✅ 可以（但约定不要）	“我希望你不要直接访问”——类外可以访问，但被认为是 内部使用
self.__is_on	私有属性	❌ 不能直接访问	Python 会做 name mangling，真正变量名变成 <i>ClassName_is_on</i> ，防止外部访问

你送朋友一个 📁 礼盒 GiftBox，里面有一些礼品：

- 🍫 一盒巧克力 → 朋友可以**直接取出**
- 📧 一个信封（写着“私人”）→ 朋友**可以打开，但不推荐**
- 🗝️ 一个上锁的小盒子 → 朋友**无法打开**，除非破解

封装 (Encapsulation)

定义：将数据和方法包装在一个类中，并隐藏内部实现细节

- 目的：防止外部直接访问和修改属性，增强安全性与代码可维护性
- Python 写法：使用 `_` 或 `__` 定义私有属性

```
class Lamp:
    def __init__(self, color):
        self.color = color    # 公有
        self._status = False  # 受保护
        self.__is_on = False  # 私有

lamp = Lamp("red")

print(lamp.color)    # OK
print(lamp._status)  # OK, 但不推荐（受保护）
print(lamp.__is_on)  # 会报错：AttributeError

# 真正的私有变量名：
print(lamp._Lamp__is_on) # OK（Name Mangling 方式访问）
```

组合 (Composition)

 把多个对象组合起来形成更复杂的系统

- 一个对象中包含其他对象作为属性
- 常用于模拟“部件组成整体”的结构关系

```
class LED:
    def __init__(self, color):
        self.light = color

class LampSystem:
    def __init__(self):
        self.red = LED("红灯")
        self.green = LED("绿灯")
        self.blue = LED("蓝灯")
```

课程标题 & 学习目标

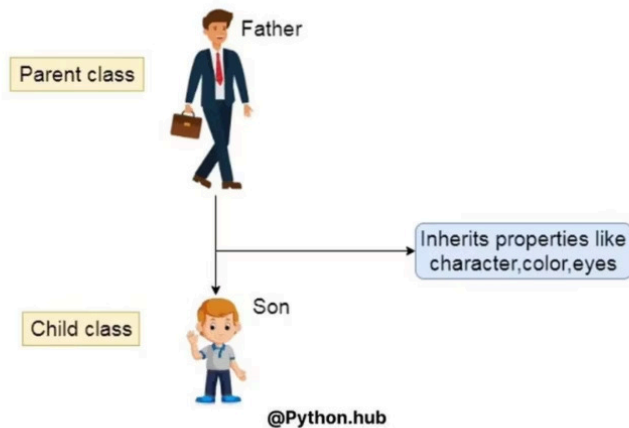
- a. 理解继承（Inheritance）概念与语法
- b. 掌握方法重写（Override）技巧
- c. 理解并实现多态（Polymorphism）行为
- d. 使用抽象基类（ABC）统一接口规范

继承 (Inheritance)

- 新类继承已有类，复用已有代码
- 例如：“导盲犬”继承“狗”，自动拥有“狗”的属性和行为，还可以新增功能

INHERITANCE

Inheritance means it allows classes to inherit common properties from the parent class.

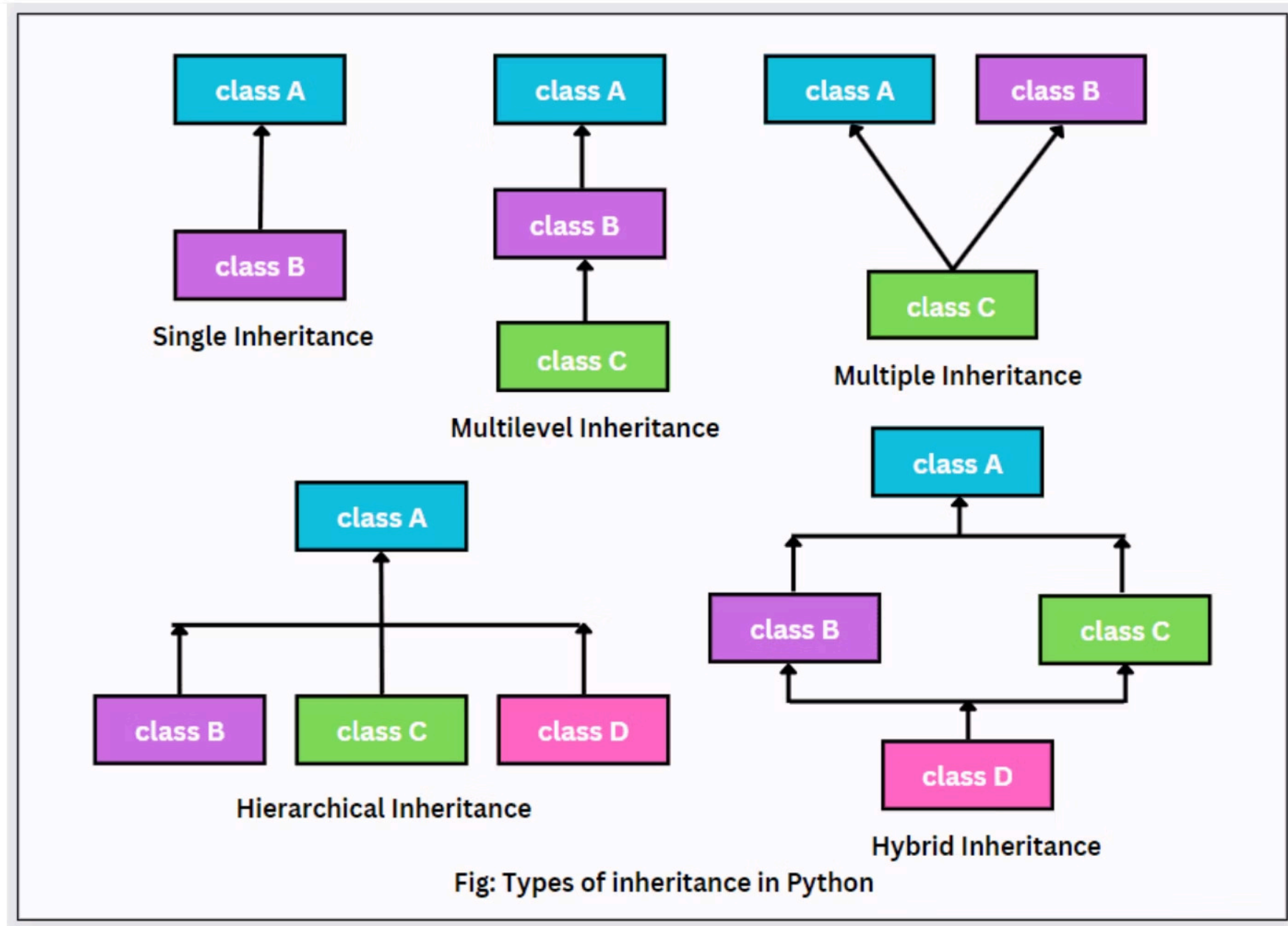


```
class Shape:
    def area(self):
        pass
    def perimeter(self):
        pass

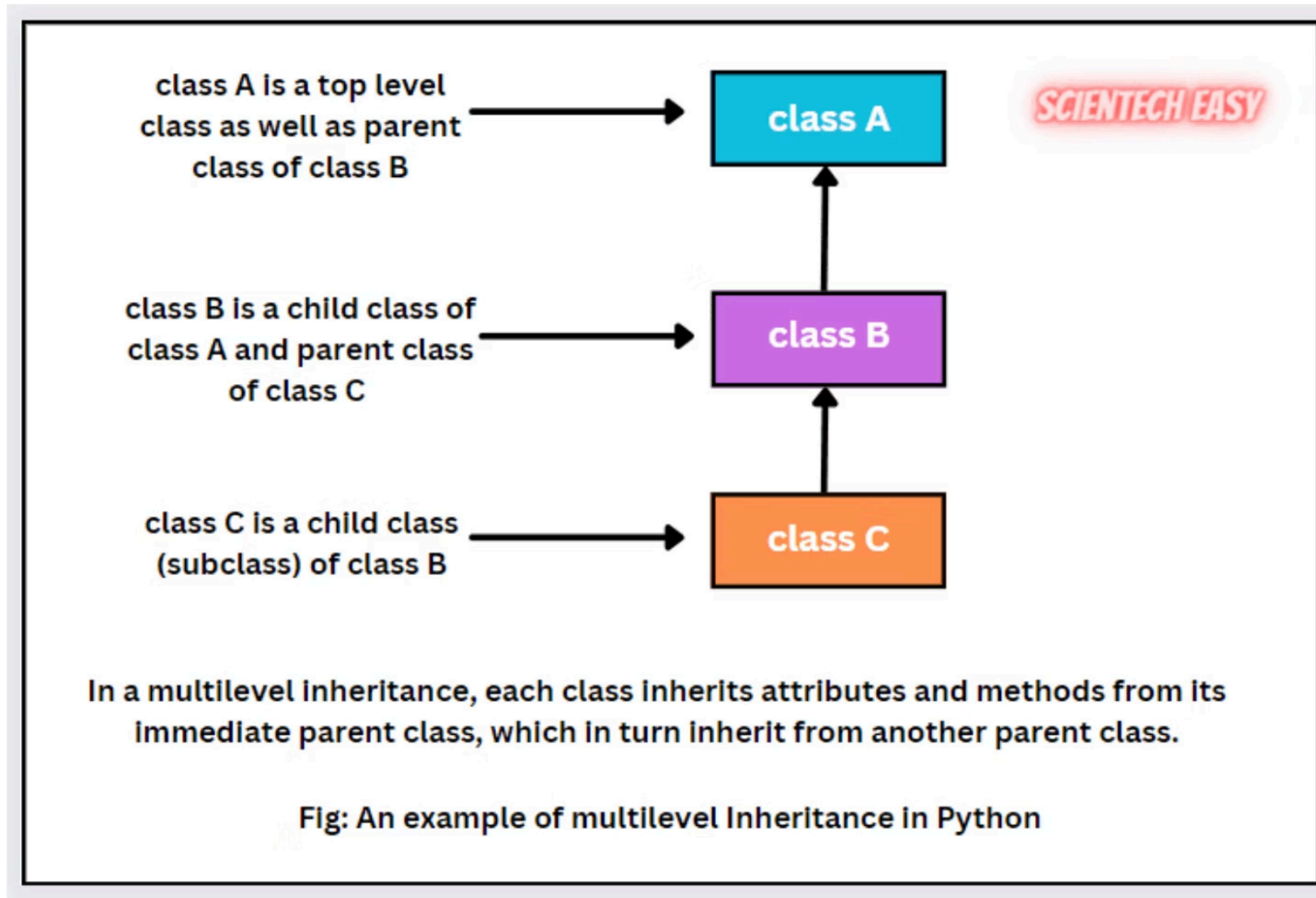
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)
```

继承种类



多层次继承



继承 (Inheritance)

继承 (Inheritance) 就像现实生活里的“父母与子女”关系：

- 子类 (child class / subclass) 可以直接拥有父类 (parent class / superclass) 的属性和方法。
- 子类也可以**重写 (override)** 父类的方法，让它的行为不同。

这样做的好处：代码复用（不用重复写相同代码）+ 结构清晰（逻辑分层）。

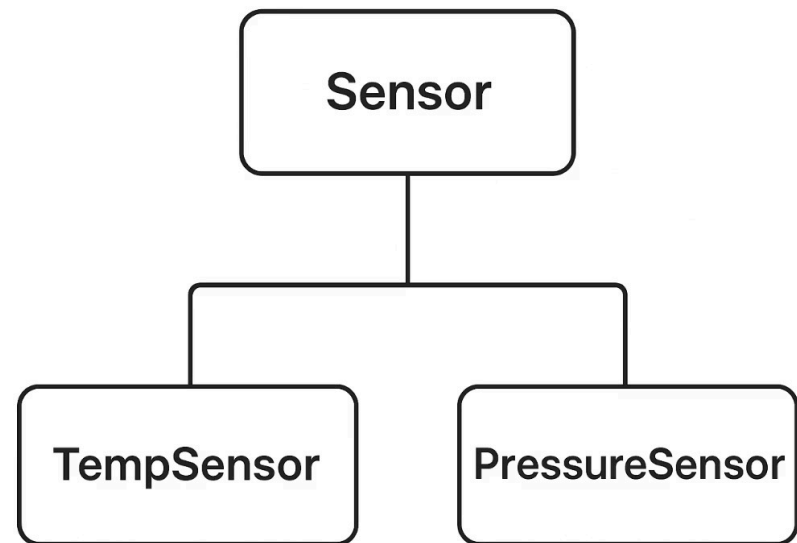
传感器示例：父类 (Sensor)

```
class Sensor:
    def __init__(self, name):
        self.name = name

    def read(self):
        return "No data"
```

- 有一个 `__init__` 构造方法，接受 `name` 参数，并保存到实例属性 `self.name`。
- 有一个 `read()` 方法，默认返回 "No data"。

可以理解为所有传感器的通用模板。



传感器示例

子类 1 (TempSensor)

```
class TempSensor(Sensor):  
    def read(self):  
        return "23.5 °C"
```

- 继承了 Sensor，所以自动拥有 name 属性和父类的结构。
- 重写了 read() 方法，让它返回温度 "23.5 °C"，而不是 "No data"。

子类 2 (PressureSensor)

```
class PressureSensor(Sensor):  
    def read(self):  
        return "101.3 kPa"
```

- 同样继承了 Sensor，有 name 属性。
- 重写了 read() 方法，返回气压 "101.3 kPa"。

传感器示例

使用示例

```
t = TempSensor("温度传感器")  
p = PressureSensor("气压传感器")  
print(t.name, "读取:", t.read())  
print(p.name, "读取:", p.read())
```

输出:

温度传感器 读取: 23.5 °C 气压传感器 读取: 101.3 kPa

继承 (Inheritance)

- TempSensor(Sensor) 表示 TempSensor 是 Sensor 的子类。
- 子类会继承父类的构造函数、属性和方法。
- 如果子类定义了与父类同名的方法（如 read()），它会覆盖父类版本，这叫方法重写。
- 如果子类没有重写 `__init__`，会自动使用父类的 `__init__`。

继承让代码更简洁，比如不需要在每个传感器类里重复写 name 相关代码。

为什么使用继承？

问题	无继承	使用继承
重复代码多	每个类重复写 <code>__init__</code>	写在父类 <code>Sensor</code> 中共享
难以扩展	每种传感器分离实现	统一结构，便于新增子类
接口不一致	方法名不统一	所有子类都用 <code>.read()</code> 方法

💡 继承使得代码结构更清晰、逻辑更一致。

父类与子类基本结构

```
class Sensor:
    def __init__(self, name):
        self.name = name

    def read(self):
        return "No data"

class TempSensor(Sensor):
    def read(self):
        return "23.5°C"

class PressureSensor(Sensor):
    def read(self):
        return "101.3kPa"
```

- 子类自动拥有 `__init__()` 方法（除非自己重写）
- 可以重写 `read()` 来提供特定行为

方法重写 (Override)

```
class LoggerSensor(Sensor):  
    def read(self):  
        value = super().read()  
        print(f"{self.name}读取：{value}")  
        return value
```

- `super()` 调用父类方法，保留原逻辑
- 子类可扩展或替换方法内容

多态（Polymorphism）

什么是多态？

- 多态就是“同样的调用形式，可以根据对象的不同而表现出不同的行为”。
- 不同类对象用同一接口调用，表现不同的行为

```
from abc import ABC, abstractmethod

# 定义接口（抽象基类）
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

# 猫类
class Cat(Animal):
    def __init__(self, noise):
        self.noise = noise

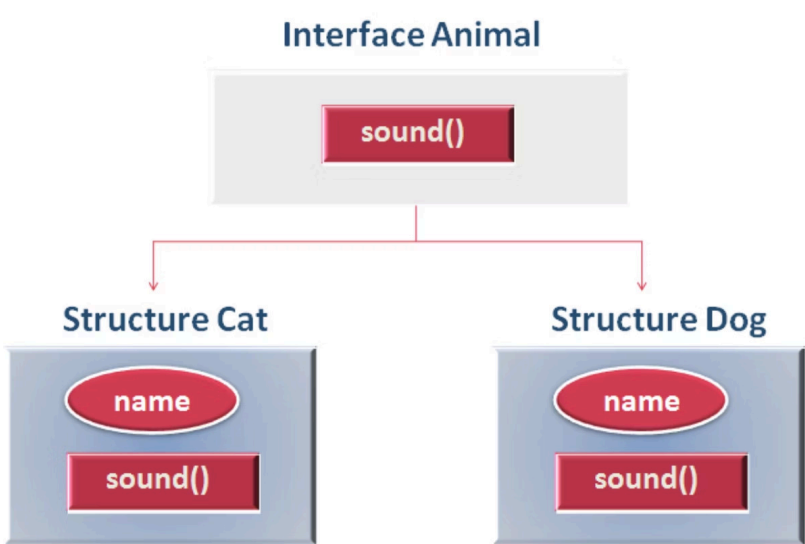
    def sound(self):
        print("Cats", self.noise)

# 狗类
class Dog(Animal):
    def __init__(self, noise):
        self.noise = noise

    def sound(self):
        print("Dogs", self.noise)

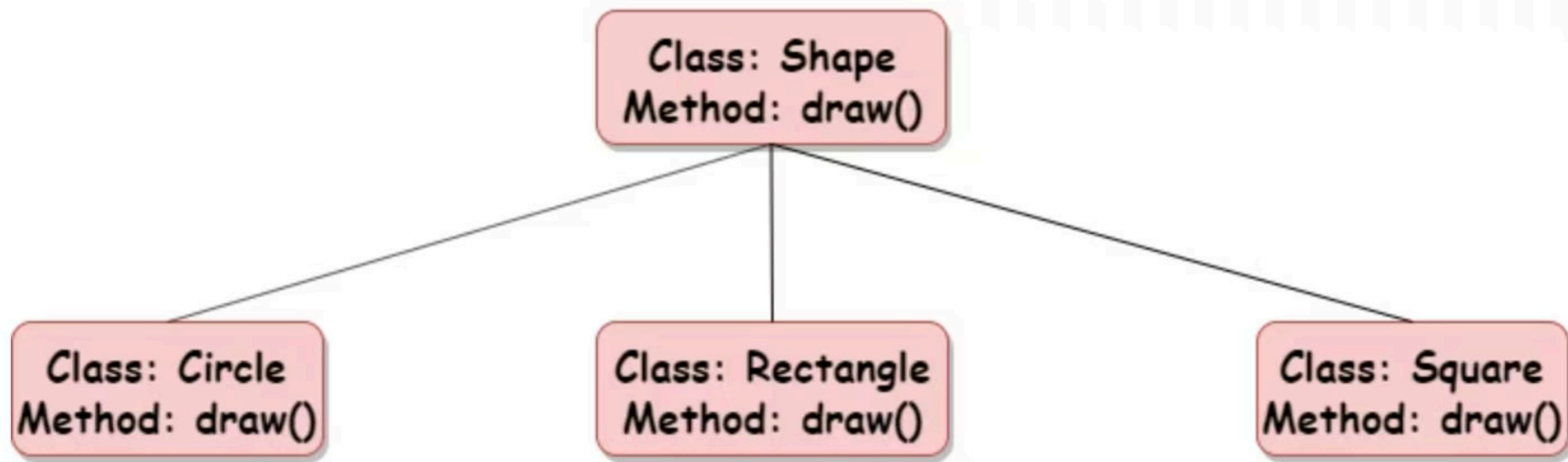
# 主函数
if __name__ == "__main__":
    # 多态：变量类型是 Animal，但可以指向不同的具体类
    animal_cat = Cat(noise="Meao")
    animal_cat.sound()

    animal_dog = Dog(noise="Bark")
    animal_dog.sound()
```



- 例如：“狗”和“猫”都能“叫”，但叫声不同

多态示例

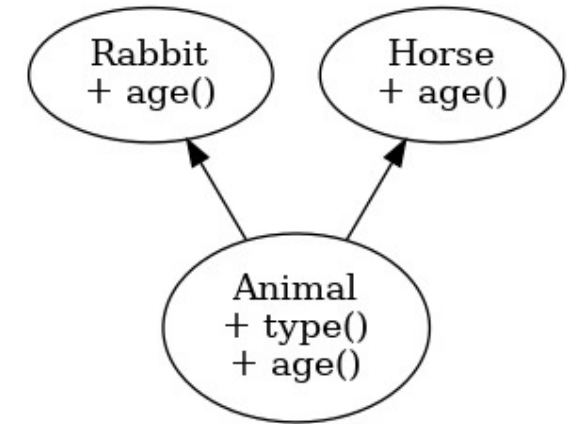


Polymorphism In Python

多态示例

```
class Animal:
    def type(self):
        print("Various types of animals")

    def age(self):
        print("Age of the animal.")
```



```
class Rabbit(Animal):
    def age(self):
        print("Age of rabbit.")
```

```
class Horse(Animal):
    def age(self):
        print("Age of horse.")
```

```
obj_animal = Animal()
obj_rabbit = Rabbit()
obj_horse = Horse()
```

```
obj_animal.type()
obj_animal.age()
```

```
obj_rabbit.type()
obj_rabbit.age()
```

```
obj_horse.type()
obj_horse.age()
```

多态的实现方式

在 Python 里，多态通常通过 继承 + 方法重写（override） 实现：

- 两个子类都继承了 Sensor。
- 但它们分别重写了 read() 方法，返回各自的数据。

```
class Sensor:
    def read(self):
        return "No data"

class TempSensor(Sensor):
    def read(self):
        return "23.5 °C"

class PressureSensor(Sensor):
    def read(self):
        return "101.3 kPa"
```

多态：统一接口调用

```
def display_all(sensors):  
    for s in sensors:  
        print(f'{s.name} -> {s.read()}')  
  
s1 = TempSensor("温度传感器")  
s2 = PressureSensor("气压传感器")  
sensor_list = [s1, s2]  
display_all(sensor_list)
```

- s.read() 会自动根据 s 的真实类型调用对应的方法：
 - 如果 s 是 TempSensor → 调用温度的 read()
 - 如果 s 是 PressureSensor → 调用气压的 read()
- 函数不需要 if 判断类型，这就是 **面向对象的多态优势**。

- 多态使得调用者无需知道对象真实类型，只要方法一致即可调用。
- 在这个例子中，display_all 不关心传入的是 TempSensor 还是 PressureSensor，只要它们有 .name 属性和 .read() 方法，就可以正常工作。
- 这种写法让代码 更通用、更灵活，不用为每种传感器写单独的循环。

✓ 函数面向父类编程，支持所有子类对象

抽象基类（ABC） 定义接口

```
from abc import ABC, abstractmethod
```

```
class AbstractSensor(ABC):
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    @abstractmethod
```

```
    def read(self):
```

```
        pass
```

- 无法直接实例化 `AbstractSensor`
- 所有子类必须实现 `read()` 方法，否则报错

💡 这是强制约定接口的一种方式

子类实现 ABC 接口

```
class TempSensor(AbstractSensor):  
    def read(self):  
        return "24.1°C"
```

```
class PressureSensor(AbstractSensor):  
    def read(self):  
        return "100.8kPa"
```

- 所有子类都有 `.read()`，可统一调用

实战应用：传感器管理器

```
class SensorManager:
    def __init__(self, sensors):
        self.sensors = sensors

    def show_all(self):
        for sensor in self.sensors:
            print(sensor.read())

sensors = [TempSensor("T1"), PressureSensor("P1")]
mgr = SensorManager(sensors)
mgr.show_all()
```

封装传感器集合，统一读取与管理，便于系统集成

小结与思考题

本节回顾：

- 继承 提供结构复用与统一接口
- 方法重写 允许子类自定义行为
- 多态 实现一致调用方式
- 抽象类 让结构更标准化

思考题：

1. 如果想增加 HumiditySensor，要做什么？
2. 如果想统一添加时间戳怎么办？
3. SensorManager 能否扩展为记录日志？

课后作业：灯的继承系统

任务描述：使用继承重构灯的系统

要求：

1. 编写父类 `BaseLamp`：包含颜色、开关状态、`toggle()`、`status()` 方法
2. 编写 3 个子类：`RedLamp`, `GreenLamp`, `BlueLamp`，重写 `status()` 方法，输出不同语言或格式
3. 编写 `LampController` 类管理多个灯，能统一调用所有灯的 `status()`

示例格式：

```
红灯状态：开  
绿灯状态：关  
蓝灯状态：开
```

进阶挑战：

- 用抽象基类 `AbstractLamp` 定义 `toggle()` + `status()` 接口
- 添加 `time_on` 属性记录每盏灯累计点亮时间（用 `datetime` 实现）